
Automatic solving of combinatorial problems using LLMs and constraint programming

Student	Oussama Belhout
Student ID	22410451
Programme	M1 Informatique
Supervisor	Pr.Jean-Charles Régim
Host laboratory	I3S
Recipients	Dr.Cinzia Di Giusto, Dr.Anne-Laure Simonelli
Project period	Spring 2026

Abstract

Many real-world tasks reduce to choosing values for a set of decision variables under a list of rules: routing delivery trucks, scheduling hospital staff, allocating compute jobs to GPUs. Specialised programs called solvers handle the search efficiently, but only when the problem is first written in their own formal input language. The present work studies whether a Large Language Model (LLM) can produce that formal input from a natural-language description.

Three design choices govern such a system: the prompting technique π , the LLM m , and the agentic design α (the number of LLM calls and how they are connected). Every configuration $c = (\pi, m, \alpha)$ was run on the same ten CSPLib problems with three LLMs. The agentic-design axis was then studied through the Progressive Delegation Ablation (PDA), a four-level ladder from a single all-in-one agent (L0) to a chain of four specialised agents (L3).

Under the conditions tested, mean success rate falls monotonically across the ladder: 100% at L0, 93.3% at L1, 80.0% at L2, 66.7% at L3, with the four hard problems driving the entire decline. Five mechanisms are proposed and unified under one principle, named here the *verifier-redundancy tax*: when an exact deterministic checker is already available, adding LLM agents tends to compete with that checker rather than help it.

Keywords: Constraint Satisfaction Problems; Large Language Models; multi-agent systems; neuro-symbolic systems; ablation study; Choco solver; CSPLib.

Contents

1	Introduction	3
1.1	Why combinatorial problems matter	3
1.2	Why language models are a candidate	3
1.3	Three design choices	3
1.4	Research questions	4
1.5	Contributions	4
1.6	Outline of the report	4
2	Background	4
2.1	Constraint Satisfaction Problems	4
2.2	Large Language Models	5
2.3	The three design axes in detail	5
2.4	Self-correcting systems	6
2.5	Neuro-symbolic perspective	6
2.6	Related work on multi-agent LLM systems	7
3	Materials and Methods	7
3.1	The enumeration of design combinations	7
3.2	The agentic-design axis: Progressive Delegation Ablation	7
3.2.1	The five behaviours	7
3.2.2	The four-rung ladder	7
3.3	The prompting axis	8
3.4	The model axis	8
3.5	The self-correction loop	9
3.6	Enumeration procedure	9
3.7	Benchmark suite	9
3.8	Metrics	9
3.9	Tools and reproducibility	10
4	Results	10
4.1	The configuration grid	10
4.2	Variation along the delegation axis	10
4.3	Variation along the model axis	11
4.4	Hard-tier breakdown	11
4.5	The cost dimension	12
4.6	Verdict on the pre-registered predictions	12
5	Discussion	12
5.1	Identifying the best configuration	12
5.2	Five mechanisms behind the decline	13
5.3	The verifier-redundancy tax	13
5.4	Relation to prior literature	13
5.5	Limitations	14
6	Conclusion	14
6.1	Summary	14
6.2	Future work	14

1 Introduction

1.1 Why combinatorial problems matter

Many of the hardest problems in modern technology start with a short description and explode into something no human can solve by hand. Routing a fleet of delivery trucks across a city, scheduling staff in a hospital, assigning tasks to a team of robots, allocating compute jobs to a cluster of GPUs, and laying out the transistors of a microchip all share the same underlying mathematical structure: choose values for a set of decision variables under a list of rules.

The difficulty is not the description; the difficulty is the size of the choice space. A distribution of twenty employees over seven shifts admits more than 10^{20} different assignments. Listing them one by one is impossible, even on a fast computer. Special-purpose programs called *solvers* can search such choice spaces without listing every candidate [1]. They form the engine behind airline scheduling, factory planning, and chip design.

A solver, however, does not accept everyday language as input. It accepts only a *formal model*: a precise description written in the solver’s own input language. Writing the formal model is the real bottleneck. The task requires both mathematical training and detailed familiarity with the specific solver in use. Such expertise remains scarce. The motivation of the present work is to replace the human modeller by an automated component.

1.2 Why language models are a candidate

A Large Language Model (LLM) is a neural network trained on very large amounts of text. It accepts a problem stated in natural language, that is, the way a human would describe it, and produces text in response, including code. An LLM is therefore a plausible candidate to translate a natural-language problem statement into the formal model that a solver requires [3].

The role of the LLM in the system studied here is intentionally limited. The LLM produces the formal model. The actual search through the choice space is performed by the solver. The LLM never explores the space itself, because LLM outputs are *probabilistic* and offer no guarantee of correctness. The solver, in contrast, is *deterministic*, in the strict sense: identical input produces identical output on every run. The task dispatch between a probabilistic component (the LLM) and a deterministic component (the solver) is the defining feature of *neuro-symbolic* systems [7], described in Section 2.5.

1.3 Three design choices

Building such a system requires three design choices.

The first design choice is the *prompting technique*, denoted π . A prompting technique is the template used to phrase the request given to the LLM. The template may contain instructions, worked examples, and conditions to respect on the expected output format.

The second design choice is the *LLM* itself, denoted m . Different LLMs differ in size, in the data on which they were trained, and in their reliability when generating code.

The third design choice is the *agentic design*, denoted α . An *agent*, in this report, is one LLM call configured to perform one specific task. An agentic design specifies how many agents the system contains and how the output of each agent is passed to the next.

A configuration of the system is denoted by the triple

$$c = (\pi, m, \alpha) \in \mathcal{C} = \Pi \times M \times A,$$

where Π , M , and A are the sets of prompting techniques, LLMs, and agentic possible designs that are considered. The three design choices combine to form the final result. A stronger LLM can compensate for a weaker prompting technique. A deeper agentic design can amplify a small per-agent error rate. Varying only one of the three at a time would mix effects together; varying

all three jointly and recording the outcome for every combination makes it possible to attribute each observed effect to the right design choice.

One further property makes the present study particularly attractive. The combinatorial solver under consideration is generic: the same solver handles 4-Queens, scheduling, Sudoku, and graph colouring without modification. Specialised algorithms exist for each of these problems, but every such algorithm only solves the one problem it was designed for. A successful natural-language interface to a generic solver therefore unlocks many problems at once, instead of one problem at a time.

1.4 Research questions

Three research questions guide the study.

- **RQ1.** How can LLMs be used to solve Constraint Satisfaction Problems?
- **RQ2.** Which configuration $c \in \mathcal{C}$ produces the highest success rate on the benchmark problems used here?
- **RQ3.** Can the system correct its own errors without human intervention when a generated model fails to compile or to satisfy the problem?

1.5 Contributions

This work makes four contributions.

1. An *enumeration of all design combinations*: every triple (π, m, α) in the configuration space $\mathcal{C}$ was run on the same benchmark suite, with the same metrics recorded for each cell. The enumeration provides a direct, measured answer to **RQ2**.
2. A focused study of the agentic-design axis, named the *Progressive Delegation Ablation* (PDA). The word *ablation* here means a controlled experiment in which one design choice (the number of agents) is varied while the other two are held fixed. The PDA compares four agentic designs $\alpha \in \{\text{L0}, \text{L1}, \text{L2}, \text{L3}\}$, in which L0 uses a single agent and L3 uses a chain of four specialised agents.
3. A measured finding that contradicts a common assumption in the multi-agent-LLM literature [5, 6]. On the benchmark suite tested here, the mean success rate decreases monotonically from 100% at L0 to 66.7% at L3 as the number of agents grows.
4. An explanation of why the decline occurred, composed of five mechanisms (M1–M5) and unified under one named principle, the *verifier-redundancy tax*.

1.6 Outline of the report

Section 2 explains the background that the rest of the report relies on: what a CSP is, what an LLM is, and what a neuro-symbolic system is. Section 3 describes how the experiment was set up. Section 4 reports the numbers obtained. Section 5 explains the numbers and proposes the verifier-redundancy tax. Section 6 concludes and lists future directions.

2 Background

2.1 Constraint Satisfaction Problems

A Constraint Satisfaction Problem (CSP) is described by three components: the *decision variables*, the values each variable may take, and the rules the variables must obey. Formally, a CSP is a triple $\mathcal{P} = (V, D, C)$, where $V = \{v_1, \dots, v_n\}$ is the set of variables, $D = \{D_1, \dots, D_n\}$ assigns to each variable v_i a finite set D_i of admissible values, and C is the set of constraints. A *solution* of $\mathcal{P}$ is an assignment of one value from D_i to each v_i such that every constraint in C is satisfied.

A small example is the 4-Queens problem, illustrated in Figure 1 (an instance of CSPLib problem 054 [2]). Four queens must be placed on a 4×4 chessboard such that no two queens attack each other. Each variable v_i stands for the column position of the queen in row i . Each domain D_i is the set $\{1, 2, 3, 4\}$ of possible columns. The constraints state that no two queens share a column ($v_i \neq v_j$ for $i \neq j$) and that no two queens share a diagonal ($|v_i - v_j| \neq |i - j|$). The placement $(v_1, v_2, v_3, v_4) = (2, 4, 1, 3)$ satisfies every constraint.

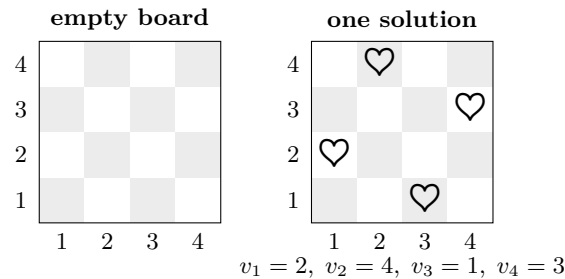


Figure 1: The 4-Queens problem. Left: the empty 4×4 board. Right: one valid placement, expressed as values for the four variables $v_1, \dots, v_4$, where v_i is the column of the queen placed in row i . Rows are numbered top to bottom; columns left to right.

A solver such as Choco [1] searches the space of assignments without listing every candidate. The solver alternates two techniques. The first technique is *backtracking*: when a partial assignment cannot be completed without violating a constraint, the solver undoes its most recent choice and tries another value. The second technique is *constraint propagation*: after each choice, the solver removes from the remaining domains the values that can no longer satisfy the constraints. Choco is deterministic with respect to its input: identical Java input produces identical output and identical search statistics on every run.

2.2 Large Language Models

A Large Language Model is a neural network trained on very large amounts of text. It produces text one piece at a time. The pieces are short character sequences, called *tokens*. The LLM works by predicting which token is most likely to come next, given the tokens it has already read.

Three properties of modern LLMs matter for the present work. The first property is that an LLM can only read a limited amount of text in a single call. The maximum is called the *context window*. The second property is that an LLM accepts formatted instructions, including worked examples placed directly inside the prompt. The LLM learns from those examples without being retrained, a property usually called *in-context learning* [3]. The third property is that the LLM can be constrained to produce structured output, for example a JSON object with named fields and typed values, when the structure is requested explicitly in the prompt.

Two limitations of LLMs are equally important. The first limitation is that the LLM can produce output that looks correct but means something different from what is intended. This kind of error is called a *silent semantic error*: silent because nothing visibly fails, semantic because the meaning is wrong. The second limitation is that small changes in the prompt can produce noticeably different outputs. These two limitations are the reason the system studied here relies on an external solver to decide correctness (Section 2.1), and on a self-correction loop to recover from errors (Section 2.4).

2.3 The three design axes in detail

The three design choices (π, m, α) introduced in Section 1.3 are revisited here in more detail.

The prompting technique π specifies how the request is phrased to the LLM. A typical prompt has three blocks: a *system instruction* that states what the LLM should do, one or more *worked*

examples that show an input together with the expected output, and the *actual user request* for which an output is wanted. Figure 2 shows a simplified formaliser prompt structured in this way. Three families of prompting recur in this work. The first family is *few-shot prompting* [3]: one or two worked examples are placed directly in the prompt. The second family is *Chain-of-Thought prompting* [3]: the prompt contains a directive that asks the LLM to write its reasoning step by step before stating a final answer. The third family is *Reflexion-style prompting* [4]: the LLM is shown the failure trace of a previous attempt and is asked to revise its output accordingly.

```
[System instruction]
You are a constraint-modelling assistant. Given a problem in natural
language, return a JSON object with three fields: variables, domains,
constraints.

[Worked example]
Input: "Place 4 queens on a 4x4 board so that no two attack each other."
Output:
{
  "variables": ["v1", "v2", "v3", "v4"],
  "domains": {"v1": [1,2,3,4], ..., "v4": [1,2,3,4]},
  "constraints": ["all_different(v1,v2,v3,v4)",
                  "no_two_on_same_diagonal(v1,...,v4)"]
}

[Actual user request]
Input: "Place 8 queens on an 8x8 board so that no two attack each other."
Output: <LLM completes here>
```

Figure 2: Anatomy of the formaliser prompt. A system instruction states the task, a worked example shows the expected input–output shape (here for 4-Queens), and the actual request follows. The LLM completes the empty Output field below the request.

The model axis m selects the specific LLM. Models differ in how many parameters they contain, in the data on which they were trained, and in whether they have been fine-tuned to produce explicit reasoning traces. These differences affect both per-call accuracy and per-call response time.

The agentic-design axis α specifies how many agents the system contains and how the output of one agent is passed to the next. The simplest agentic design uses a single agent for the whole task. A deeper agentic design splits the task into sub-tasks and assigns each sub-task to a dedicated agent. The four agentic designs studied in this work are described in Section 3.2.

2.4 Self-correcting systems

A self-correcting system contains a loop that reads its own failure signals and tries again [4]. A *failure signal* can be a compilation error from the Java compiler, a runtime exception from the solver, or a complaint from an internal LLM verifier. When a failure signal appears, the system re-invokes the LLM with the failure trace attached as additional context, and the LLM produces a new attempt. The new attempt is submitted to the solver again. The loop terminates when an attempt succeeds, or when a fixed retry budget is exhausted. This mechanism provides the operational answer to **RQ3**: within its retry budget, the system recovers from typical errors without human intervention.

2.5 Neuro-symbolic perspective

The system studied here is an instance of a wider class called *neuro-symbolic* [7]. A neuro-symbolic system pairs a *neural* component that reasons probabilistically over natural language (here the LLM) with a *symbolic* component that manipulates exact logical or mathematical objects (here the Choco solver). The specific task that the two components solve together is called the *business*

logic of the system: clinical decision rules in a medical-diagnosis chatbot, statutes and case law in a legal-advice assistant, the production of a Choco model from a natural-language CSP description in the present work.

The recurring engineering question across these systems is the same: how should the probabilistic part and the exact part be coupled so that the exact part remains the source of truth [8]? The verifier-redundancy tax of Section 5.3 is one partial answer for the case of CSP solving.

2.6 Related work on multi-agent LLM systems

Several frameworks split a single user request across multiple LLM agents that exchange messages. AutoGen [5] provides a generic conversational scaffold. MetaGPT [6] and similar role-specialisation frameworks (ChatDev and related systems) simulate a software-engineering team with explicit role specialisation and report measurable gains over single-agent baselines on tasks such as code writing, design, and debate.

One property of those tasks deserves attention. None of them includes an automatic, exact external checker that decides correctness without ambiguity [8]. The present study evaluates the same multi-agent idea on a task that *does* include such a checker (the Choco solver). The contrast between the two settings is the heart of Section 5.

3 Materials and Methods

3.1 The enumeration of design combinations

The configuration space $\mathcal{C} = \Pi \times M \times A$ defined in Section 1.3 was enumerated cell by cell. For each configuration $c \in \mathcal{C}$, the system was run on the same benchmark suite (the set of problems), and the same metrics were recorded. The cells were then compared on equal footing. The cell with the highest success rate provides the surface-level answer to **RQ2**. The shape of the response surface along each axis identifies which design choice drives that answer. The agentic-design axis is studied in particular depth in Sections 3.2 and 4.2.

3.2 The agentic-design axis: Progressive Delegation Ablation

3.2.1 The five behaviours

Converting a natural-language CSP description into a verified solution requires five distinct behaviours. Every configuration on the ladder performs all five behaviours. What changes across configurations is the way the five behaviours are distributed across one or more agents.

- **Formalisation:** read the natural-language description and extract the variables, their domains, and the constraints.
- **Modelling:** generate Java code that expresses the formalisation against the Choco API.
- **Validation:** inspect the generated model before the solver is invoked, and flag any obvious mismatch with the original problem statement.
- **Refinement:** read the failure message returned by the compiler, the validator, or the solver, and produce a corrected model.
- **Explanation:** produce a textual description of the returned solution, expressed using the variables of the original problem.

3.2.2 The four-rung ladder

The PDA defines four agentic designs, denoted L0, L1, L2, and L3. The four designs are built by successive specialisation. L0 contains a single agent (the *monolith*) that performs all five

behaviours. At each higher rung, one behaviour is detached from the monolith and given to a new dedicated agent. The agent-to-behaviour assignment is summarised in Table 1 and illustrated in Figure 3.

Table 1: The four-rung delegation ladder. F = Formalisation, M = Modelling, V = Validation, R = Refinement, E = Explanation.

Level	Agents	F	M	V	R	E
L0 (Monolith)	1	Monolith	Monolith	Monolith	Monolith	Monolith
L1 (Refiner split)	2	Monolith	Monolith	Monolith	Refiner	Monolith
L2 (Validator split)	3	Monolith	Monolith	Validator	Refiner	Monolith
L3 (Formaliser split)	4	Formaliser	Modeller	Validator	Refiner	Modeller

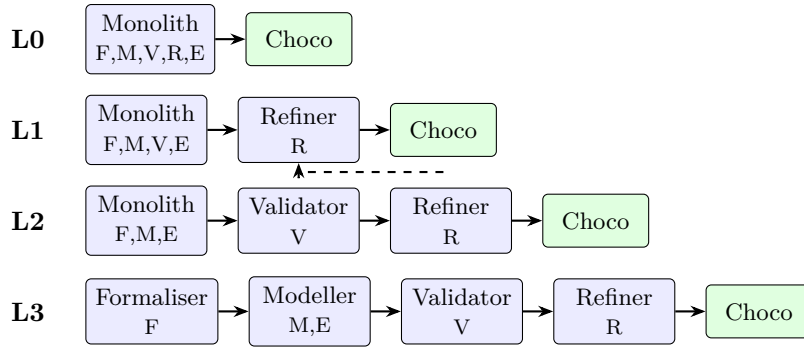


Figure 3: The four agentic designs of the PDA. The same five behaviours (F, M, V, R, E) are performed at every level; only the distribution across agents changes. The Refiner is invoked whenever a downstream component (compiler, validator, or solver) returns a failure signal; the dashed feedback edge is shown only at L1 for compactness, and exists at every level.

The hypothesis tested by the PDA is straightforward. Splitting the five behaviours across specialised agents is expected to reduce the amount of work each agent does. A smaller per-agent workload is, in turn, expected to raise per-agent reliability. The aggregate effect should be a higher end-to-end success rate at L3 than at L0, together with improvements on the cost-side metrics (wall time and refinement iterations). This hypothesis is the operational form of the assumption that underlies multi-agent frameworks such as AutoGen [5] and MetaGPT [6].

3.3 The prompting axis

Each agent on the ladder was given a prompting technique chosen for the behaviour the agent performs. The monolith uses few-shot prompting [3] with two worked examples, because few-shot prompting is the most general technique and the monolith performs the broadest set of behaviours. The refiner uses a Reflexion-style prompt [4], because the refiner must read a previous failure and adapt its output, and Reflexion is the prompting family designed precisely for that purpose. The validator uses adversarial Chain-of-Thought prompting [3], because the validator must look for defects, and adversarial Chain-of-Thought encourages the LLM to enumerate possible defects before stating a verdict. The formaliser uses structured output through a Pydantic schema, because the output of the formaliser is consumed directly by the next agent, and must therefore be parsable without ambiguity.

3.4 The model axis

Three LLMs were selected to cover deliberately different profiles. Qwen 2.5-Coder-32B was chosen because it is an open model fine-tuned on code, and therefore serves as a code-generation baseline.

Claude Haiku 4.5 was chosen because it is a small commercial model with low response time and high single-call accuracy on short instructions. DeepSeek-R1-Distill-Llama-70B was chosen because it has been trained with explicit reasoning traces, which makes it possible to test whether reasoning-oriented training compensates for delegation depth. All three models were accessed through OpenRouter, a network gateway that exposes models from many providers behind a single uniform interface. OpenRouter was chosen because identical request semantics across providers remove one source of measurement variability.

3.5 The self-correction loop

The self-correction loop is present at every level of the PDA. The loop is triggered by three failure conditions. The first condition is a compilation error reported by Maven on the generated Java code. The second condition is a runtime exception thrown by Choco during search. The third condition is the absence of a valid solution after the solver has finished. When any of the three conditions occurs, the refining behaviour is invoked with the corresponding failure trace as additional context. The refiner produces a new model, and the new model is re-submitted to Choco. The loop terminates when Choco accepts the model, or when the retry counter reaches the constant `MAX_REFINEMENT_ITERATIONS`, set to three for every experiment. The loop is the operational answer to **RQ3**; its convergence behaviour is reported in Section 4.5.

3.6 Enumeration procedure

The configurations were enumerated by a triple-nested loop, shown below. The outer loop iterates over the four delegation levels. The middle loop iterates over the three LLMs. The inner loop iterates over the ten benchmark problems. The total number of runs is therefore $|A| \times |M| \times |B| = 4 \times 3 \times 10 = 120$.

```
for level in {L0, L1, L2, L3}:
    workflow = build_workflow(level)
    for model in {Qwen, Haiku, DeepSeek}:
        for problem in benchmarks:
            run = workflow.solve(problem, model)
            record_metrics(run)
```

3.7 Benchmark suite

Ten CSP instances were drawn from CSPLib [2], the standard benchmark library used in the constraint-programming community. The instances include 4-Queens and 8-Queens (CSPLib problem 054), Magic Square (problem 019), SEND + MORE = MONEY (problem 023), Graph Colouring (problem 004), Latin Square (problem 003), and Sudoku (problem 028). The remaining instances cover a scheduling problem, a knapsack problem, and a cryptarithmic variant. The suite is partitioned into three difficulty tiers (easy, medium, hard). The hard tier contains four instances and is the tier where the effects along the agentic-design axis appear (Section 4.4). Reusing CSPLib problems rather than designing new ones was chosen because CSPLib makes the present results directly comparable with prior constraint-programming work.

3.8 Metrics

Each metric is tied to one or more research questions. The *success rate* (the fraction of problems for which Choco returned a valid solution) addresses **RQ1** and **RQ2**. The *first-shot success rate* (the fraction of problems solved at iteration zero, before any refinement was needed) and the *mean number of refinement iterations* address **RQ3**. The *compilation-failure rate* (the fraction of runs in which the generated Java did not compile) addresses **RQ1**. The *wall time per problem* and the Choco *solver statistics* (nodes explored, backtracks, propagations) inform the cost analysis

of **RQ2**. A regex-based *explanation-faithfulness score*, which counts how many of the solver’s variable names also appear in the textual explanation produced by the LLM, was also recorded. The score’s limitations are reported in Section 5.5.

3.9 Tools and reproducibility

The symbolic component is Choco Solver [1], a Java constraint-programming library. Generated Java models were compiled by Apache Maven and executed in a dedicated subprocess. The agent graphs were implemented in LangGraph, a Python library for building state-machine-style LLM workflows. LangGraph was chosen because the per-level workflow can be expressed as a graph in which each node is one LLM call and each edge is a conditional transition. LLM calls were routed through OpenRouter, as explained in Section 3.4. LangSmith was used to trace and store every LLM call. Choco is deterministic with respect to its Java input: identical generated source produces identical output and identical search statistics on every run. The only non-deterministic part of every measurement is therefore the LLM call itself.

4 Results

The results are reported along the three design axes. The success rate per cell is the headline metric. The cost-side metrics (compilation failures, refinement iterations, wall time) are reported separately. All values are computed over the ten-problem suite of Section 3.7.

4.1 The configuration grid

The success rate of each cell of the (level, model) grid is reported in Table 2. The L0 row is uniformly at 100%, independent of the LLM. The L3 row ranges between 50% and 90% depending on the LLM. The grid identifies the cells at $\alpha = L0$, for any model, as the highest-performing configurations under the tested conditions.

Table 2: Success rate per configuration cell. Rows: delegation levels. Columns: LLMs. Values are percentages over the ten-problem suite.

	Qwen 2.5-Coder-32B	Haiku 4.5	DeepSeek-R1 70B	Mean
L0	100	100	100	100.0
L1	100	100	80	93.3
L2	90	50	100	80.0
L3	90	60	50	66.7

4.2 Variation along the delegation axis

The strongest pattern in the grid lies along the agentic-design axis α . Figure 4 reports the success rate of each LLM at each level, together with the mean across the three LLMs. The mean decreases at every step: 100.0% $\rightarrow$ 93.3% $\rightarrow$ 80.0% $\rightarrow$ 66.7%. The total loss between L0 and L3 is 33.3 percentage points. Two of the three LLMs, Qwen and Haiku, also decrease individually between L1 and L3. DeepSeek alternates across levels but ends at 50% at L3, half of its L0 value.

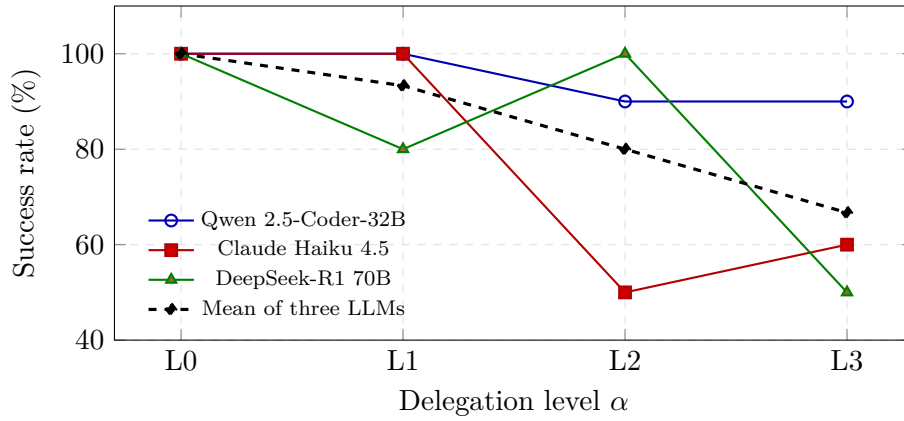


Figure 4: Success rate against delegation level α . The mean across the three LLMs decreases monotonically as the number of agents grows.

4.3 Variation along the model axis

The three LLMs exhibit three distinct response profiles along the delegation axis. Qwen 2.5-Coder-32B is the most stable: it loses ten percentage points between L1 and L2 and remains at 90% at L3. Claude Haiku 4.5 ties for the strongest LLM at L0 and L1 (100%), but falls to 50% at L2. DeepSeek-R1 70B is the only LLM whose L2 value (100%) exceeds its L1 value (80%); it then drops to 50% at L3. The ranking of the three LLMs is therefore not preserved across levels. Performance at L0 does not predict performance at L3.

4.4 Hard-tier breakdown

The easy and medium tiers were solved at or near 100% across every cell. The entire effect of the delegation axis lives in the four hard-tier problems. Figure 5 reports the count of hard-tier problems solved at each level for each LLM (out of four per LLM). The total across the three LLMs falls from 12/12 at L0 to 5/12 at L3. The four hard problems that fail at the deeper levels are Sudoku, SEND + MORE = MONEY, the scheduling instance, and the knapsack instance.

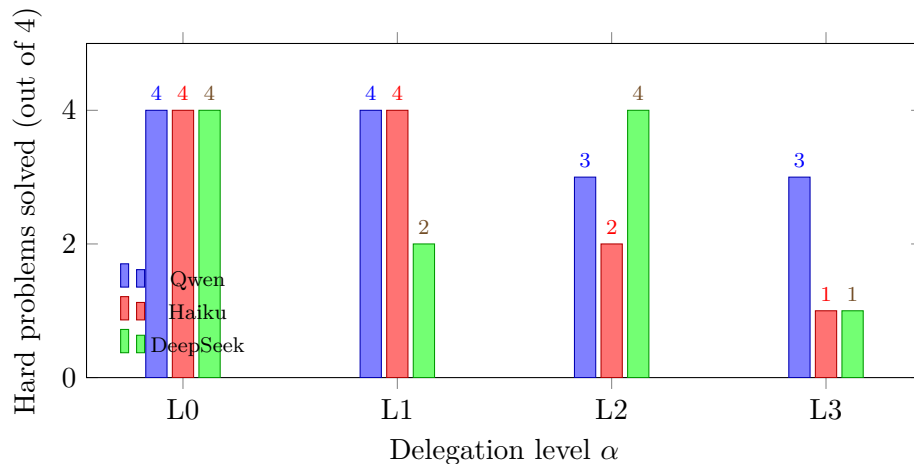


Figure 5: Hard-tier success per LLM and per level (out of four hard problems per LLM). The total across the three LLMs falls from 12/12 at L0 to 5/12 at L3.

4.5 The cost dimension

Three cost-side metrics deepen the picture. First, the compilation-failure rate rises from 0% at L0 to approximately 27% at L3. Second, the mean number of refinement iterations approximately triples between L0 and L3. Third, the mean wall time per problem grows from 47 seconds at L0 to 788 seconds at L3, a factor of sixteen.

The L3 wall-time mean is, however, distorted by a single outlier. One DeepSeek run on SEND + MORE = MONEY reported an elapsed time of 21 318 seconds. Inspection of the trace showed that the elapsed time was not produced by model reasoning, but by a stalled HTTP request to the provider, which eventually returned a timeout error. The corresponding run produced no Java code and triggered no refinement iteration. Removing this single run lowers the L3 mean wall time appreciably. The qualitative ordering of the four levels ($L0 < L1 < L2 < L3$ in wall time) is preserved with or without the outlier.

The compilation-failure trajectory of Claude Haiku 4.5 is reported separately in Figure 6. Its non-monotonic shape (0% at L0, 0% at L1, 50% at L2, 30% at L3) is the strongest evidence for mechanism M3 of Section 5.2.

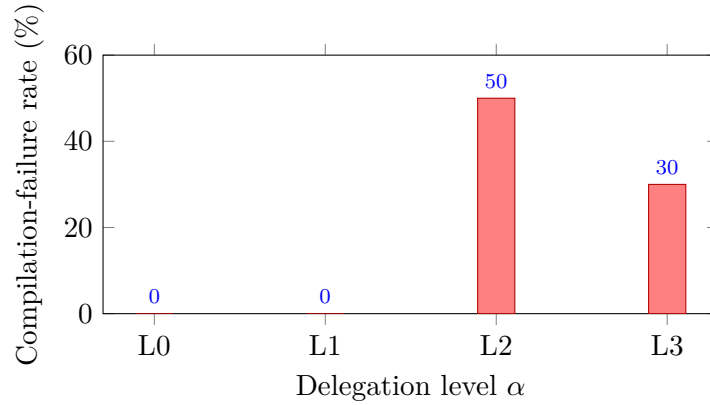


Figure 6: Claude Haiku 4.5: compilation-failure rate per delegation level. The jump at L2 coincides with the introduction of the validator into the system.

4.6 Verdict on the pre-registered predictions

Four directional predictions were registered before the measurements were taken. The predictions and the observed outcomes are reported in Table 3. All four predictions were refuted by the data.

Table 3: Pre-registered predictions and observed outcomes.

Rung	Prediction	Observation
L1 vs. L0	Mean success up	Mean success down (100 $\rightarrow$ 93.3)
L2 vs. L1	Mean success up; iterations down	Both refuted
L3 vs. L2	Mean success up on hard tier	Hard tier 9/12 $\rightarrow$ 5/12

5 Discussion

5.1 Identifying the best configuration

On the benchmark suite tested, the configurations with the highest success rate are those at the shallowest delegation level. The L0 row of Table 2 is uniformly at 100%, whatever the LLM. The surface-level answer to **RQ2** is therefore the monolithic configuration. The remaining question is why the deeper configurations, which the multi-agent-LLM literature treats as beneficial, fail to

improve upon L0 on the present task. The rest of Section 5 addresses that question through five mechanisms (Section 5.2) and one unifying principle (Section 5.3).

5.2 Five mechanisms behind the decline

Five mechanisms are proposed. Each mechanism is tied to a specific datum reported in Section 4.

M1 – Errors stack up across stages. A system composed of k sequential stages, each correct with probability p , has joint correctness bounded above by p^k . With $p = 0.95$ and $k = 4$, the bound is approximately 0.81. M1 alone accounts for a 19 percentage-point gap between L0 and L3. The observed mean gap is 33.3 percentage points. The remaining 14 points must therefore come from other mechanisms.

M2 – Redundancy with the external checker. Choco is a deterministic and exact checker: either the generated model compiles and yields a valid solution, or it does not. Placing a probabilistic LLM critic above an exact checker adds noise to the decision without adding information. The expected contribution of the critic is at best zero, and becomes negative whenever the critic flags correct code as faulty.

M3 – The critic introduces its own bugs. A prompt that asks the validator to find defects rarely returns an empty answer. The validator therefore tends to flag correct code. The refiner, which is instructed to trust the validator, then modifies the flagged code. The modification can introduce errors that were not present in the original model. The strongest evidence is the compilation-failure trajectory of Claude Haiku 4.5 in Figure 6. Haiku produced no compilation failures at L0 or L1, but produced compilation failures on half of its runs at L2 once the validator was added. Half of the L2 compilation failures were therefore introduced by the loop itself rather than by the original generator.

M4 – The retry budget is blind to the cost of one retry. The retry budget was fixed to three at every level. At L0, one retry corresponds to one LLM call. At L3, one retry corresponds to approximately four LLM calls, because the entire agent chain is re-traversed on each attempt. The same nominal budget therefore consumes very different amounts of compute. A budget calibrated for L0 is too small to recover the genuinely hard cases at L3, and disproportionately expensive on cases that have already failed.

M5 – LLMs react differently to delegation. The three LLMs exhibit three different decline profiles along α (Section 4.3). Haiku leads at L0 and L1, then collapses at L2. DeepSeek alternates. Qwen remains the most stable. No single ranking of the three LLMs is preserved across the four levels. The performance of an LLM on the monolithic configuration is therefore not a reliable predictor of its performance under deeper delegation.

5.3 The verifier-redundancy tax

The five mechanisms can be unified under one principle. Multi-agent decomposition is beneficial when it *substitutes* for a missing external checker, as in essay writing, code review without execution, or open-ended dialogue. Multi-agent decomposition is harmful when it *competes* with an external checker that is already exact and deterministic, as in CSP solving against a complete solver [8]. On the benchmark suite tested here, the harm amounts to a loss of approximately 7 to 13 percentage points of mean success rate per added agent. This work names that loss the *verifier-redundancy tax*, a CSP-specific instance of the broader LLM-Modulo principle of Kambhampati et al. [8]. The principle is the contribution restated from Section 1.5.

5.4 Relation to prior literature

The results presented here do not contradict prior multi-agent work. AutoGen [5], MetaGPT [6], and ChatDev evaluate decomposition on tasks for which no automatic external checker exists. On those tasks, decomposition reports gains over single-agent baselines. The present study suggests that the gains do not transfer to tasks for which an external checker already exists. The present

work supplies the missing controlled comparison: the same five behaviours, the same benchmark suite, the same exact checker, varied only along the agentic depth. The contribution of the present work is therefore to restrict, not to refute, the scope of the prior claims.

5.5 Limitations

Five limitations should be made explicit before the present results are generalised.

First, the hard tier of the benchmark contains four problems per LLM, for a total of twelve cells per level. The mean trend is monotonic, but the per-cell differences fall within one standard error. A larger benchmark would be required to estimate the per-level effect size more precisely.

Second, the Choco statistics parser produced suspiciously constant values for nodes, backtracks, and propagations across instances of very different sizes. Fine-grained comparisons of search effort across configurations were therefore not possible on the current logs.

Third, the regex-based explanation-faithfulness score returned zero on every run, because the underlying solution dictionary remained empty during logging. The score is therefore structurally undefined for the runs collected here. A different scoring protocol (for example, an LLM-as-judge protocol) would be required to evaluate explanation quality.

Fourth, per-call token counts were not persisted. The cost-decoupling argument associated with L3 cannot be quantified on the current data.

Fifth, only sequential decomposition was tested. The verifier-redundancy tax was observed for a chain topology. Whether the principle generalises to alternative topologies (debate, voting, tree-structured agentic designs) is left for future work.

6 Conclusion

6.1 Summary

This work introduces an enumeration of all design combinations for an LLM-driven CSP-solving system. Three design choices were varied jointly: the prompting technique π , the LLM m , and the agentic design α . The agentic-design axis was then studied in depth through the Progressive Delegation Ablation (PDA), which compares four system depths from a single all-in-one agent (L0) to a chain of four specialised agents (L3). On the ten-problem benchmark suite tested, the mean success rate falls monotonically from 100% at L0 to 66.7% at L3, and the four hard-tier problems account for the entire decline. Five mechanisms (M1 to M5) are proposed to explain the decline. The five mechanisms are unified under a single principle, the *verifier-redundancy tax*: multi-agent decomposition tends to compete with, rather than complement, an external checker that is already exact. The contribution of the present work is therefore both empirical (a refutation of the pre-registered hypothesis under the tested conditions) and methodological (a principle that delimits when multi-agent decomposition is expected to help).

6.2 Future work

Six directions follow directly from the limitations of Section 5.5. First, the delegation order could be reversed (formaliser first, then validator, then refiner) to test whether a different error profile emerges. Second, the validator could be allowed to abstain rather than be required to produce a critique, which would directly test mechanism M3. Third, the retry budget could be made level-aware, scaling with the per-iteration LLM-call count. Fourth, the Choco statistics parser should be corrected before any further claim on search effort is drawn. Fifth, the hard tier should be expanded to at least twelve problems per LLM to obtain adequate statistical power. Sixth, non-sequential agentic designs (debate, voting, tree-structured) should be evaluated against the verifier-redundancy tax to test the generality of the principle.

References

- [1] C. Prud'homme and J.-G. Fages, “Choco-solver: A Java library for constraint programming,” *Journal of Open Source Software*, vol. 7, no. 78, p. 4708, 2022. doi: 10.21105/joss.04708.
- [2] I. P. Gent and T. Walsh, “CSPLib: A benchmark library for constraints,” in *Principles and Practice of Constraint Programming (CP'99)*, Lecture Notes in Computer Science, vol. 1713. Berlin, Heidelberg: Springer, 1999, pp. 480–481. [Online]. Available: <https://www.csplib.org>
- [3] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022. arXiv:2201.11903.
- [4] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. arXiv:2303.11366.
- [5] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “AutoGen: Enabling next-generation large language model applications via multi-agent conversation,” arXiv preprint arXiv:2308.08155, 2023.
- [6] S. Hong, X. Zheng, J. Chen, Y. Cheng, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, and C. Wu, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.00352.
- [7] Z. Wan, C.-K. Liu, H. Yang, C. Li, H. You, Y. Fu, C. Wan, T. Krishna, Y. Lin, and A. Raychowdhury, “Towards cognitive AI systems: A survey and prospective on neuro-symbolic AI,” arXiv preprint arXiv:2401.01040, 2024.
- [8] S. Kambhampati, K. Valmeekam, L. Guan, M. Verma, K. Stechly, S. Bhambri, L. Saldyt, and A. Murthy, “LLMs can't plan, but can help planning in LLM-Modulo frameworks,” in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.01817.
- [9] F. Régim, E. De Maria, and A. Bonlarron, “Combining constraint programming reasoning with large language model predictions,” in *Proceedings of the 30th International Conference on Principles and Practice of Constraint Programming (CP 2024)*, LIPIcs, vol. 307, art. 25, 2024. doi: 10.4230/LIPIcs.CP.2024.25.
- [10] S. Szeider, “CP-Agent: Agentic constraint programming,” arXiv preprint arXiv:2508.07468, 2025.